

Literaturüberblick Teil 2: Tool Use / Function Calling / Agenten & Retrieval-Augmented Generation (RAG)

TL;DR

Tool Use und RAG sind die beiden dominierenden Augmentierungsparadigmen für LLMs und konvergieren 2024–2026 zunehmend zu "Agentic RAG": RAG liefert faktisches Wissen (Reduktion von Halluzinationen, Aktualität), Tool Use/Function Calling liefert Handlungsfähigkeit (APIs, Code, externe Systeme).

Kanonische Ankerpunkte: Für Tool Use/Agenten die Survey von Mialon et al. (2023, TMLR), Toolformer (Schick et al., NeurIPS 2023), ReAct (Yao et al., ICLR 2023) und Gorilla/BFCL (Patil et al.); für RAG das Originalpaper von Lewis et al. (NeurIPS 2020), die Survey von Gao et al. (2023) mit der Taxonomie Naive/Advanced/Modular RAG sowie Self-RAG, CRAG und GraphRAG.

Empfehlung für die Präsentation: In zwei Blöcken (A Tool Use/Agenten, B RAG) mit einer verbindenden Klammer "Agentic RAG" aufbauen; jeweils Survey-Fundament → Kerntechniken → Benchmarks → aktuelle Entwicklungen → Grenzen; die Praxis-Ecke (LangChain, LlamaIndex, MCP, Vektor-DBs) am Ende als Umsetzungsleitfaden.

Survey-Fundament

A) Tool Use / Agenten

Augmented Language Models: a Survey — Grégoire Mialon, Roberto Dessì, Maria Lomeli et al. (Meta AI), TMLR 2023, arXiv:2302.07842. Grundlegende Survey, die LLM-Augmentierung entlang zweier Achsen ordnet: *Reasoning* (Zerlegung komplexer Aufgaben in Teilaufgaben) und *Tools/Acting* (Aufruf externer Module wie Code-Interpreter). [arXiv](#)

➤ Prägt den Begriff "Augmented Language Models" (ALMs). [arXiv](#)

➤ **Tool Learning with Foundation Models** — Yujia Qin, Shengding Hu et al. (Tsinghua/OpenBMB), 2023, arXiv:2304.08354. Umfassende Systematik des "Tool Learning" mit Foundation Models; Rahmenwerk für Wahrnehmung, Planung, Ausführung von Werkzeugaufrufen.

LLM With Tools: A Survey (Zhuocheng Shen, 2024, arXiv:2409.18807) und **Tool Learning with Large Language Models: A Survey** (Changle Qu et al., 2024, arXiv:2405.17935) — jüngere Surveys, die den Stand der Forschung 2024 zusammenfassen.

B) RAG

Retrieval-Augmented Generation for Large Language Models: A Survey — Yunfan Gao, Yun Xiong, Xinyu Gao et al. (Tongji/Fudan), 2023, arXiv:2312.10997. Die meistzitierte RAG-Survey; etabliert die kanonische Taxonomie **Naive RAG → Advanced RAG → Modular RAG** und die dreiteilige Struktur *Retrieval, Generation,*

Augmentation. [arXiv](#)



Behandelt Evaluationsframeworks (RGB, RAGAS, ARES) und offene Herausforderungen.

A Survey on RAG Meeting LLMs: Towards Retrieval-Augmented Large Language Models — Wenqi Fan et al., KDD 2024. Praxisorientierte Ergänzung mit Fokus auf Architektur und Anwendungen.

Evaluation of Retrieval-Augmented Generation: A Survey — Hao Yu et al., 2024, arXiv:2405.07437.

Fokussiert speziell auf RAG-Evaluationsmethoden.

Kerntechniken / -paper

A) Tool Use / Function Calling / Agenten

Grundlegende Paradigmen

MRKL Systems — Ehud Karpas, Yoav Shoham, Amnon Shashua et al. (AI21 Labs), 2022, arXiv:2205.00445.

"Modular Reasoning, Knowledge and Language" (ausgesprochen "miracle"): neuro-symbolische Architektur, die ein LLM über einen **Router** an spezialisierte Expertenmodule (Rechner, Datenbanken, APIs) anbindet. [Portkey](#)



Von AI21 als Jurassic-X umgesetzt; [Portkey](#)



konzeptioneller Vorläufer von ReAct und LangChain.

ReAct: Synergizing Reasoning and Acting in Language Models — Shunyu Yao, Jeffrey Zhao, Dian Yu et al. (Princeton/Google), ICLR 2023, arXiv:2210.03629. Verschränkt Reasoning-Traces und Aktionen; das Modell "denkt" und "handelt" abwechselnd (z. B. Wikipedia-API-Aufrufe). Reduziert

Halluzination/Fehlerfortpflanzung gegenüber reinem Chain-of-Thought. ReAct übertrifft Imitation-/RL-Methoden auf ALFWorld und WebShop um eine absolute Erfolgsraten-Steigerung von **34 % bzw. 10 %** — bei nur ein bis zwei In-Context-Beispielen (Originalzitat: "ReAct outperforms imitation and reinforcement learning methods by an absolute success rate of 34% and 10% respectively"; auf ALFWorld erreicht ReAct 71 % Ø-Erfolgsrate vs. 45 % Act-only und 37 % BUTLER-Baseline). Fundierendes Paradigma für tool-nutzende Agenten (bereits in Teil 1 gestreift).

Toolformer: Language Models Can Teach Themselves to Use Tools — Timo Schick, Jane Dwivedi-Yu et al. (Meta AI), NeurIPS 2023, arXiv:2302.04761. Selbstüberwachtes Lernen von Tool-Nutzung: das Modell entscheidet selbst, welche API (Rechner, QA-System, Suchmaschine, Übersetzer, Kalender) wann mit welchen Argumenten aufgerufen wird [arXiv](#)



— "requiring nothing more than a handful of demonstrations for each API". Basismodell ist **GPT-J (6,7B)**, das durch Tool-Nutzung das ~26× größere GPT-3 (175B) auf mehreren Aufgaben übertrifft; die Fähigkeit emergiert erst ab ~775M Parametern.

Function Calling / strukturierte Tool-Aufrufe

Gorilla: Large Language Model Connected with Massive APIs — Shishir G. Patil, Tianjun Zhang, Xin Wang, Joseph E. Gonzalez (UC Berkeley), NeurIPS 2024, arXiv:2305.15334. Feingetuntes **LLaMA-7B-Modell**, das beim Schreiben von API-Aufrufen GPT-4 übertrifft (Originalzitat: "We release Gorilla, a finetuned LLaMA-based model that surpasses the performance of GPT-4 on writing API calls"). Im Oracle-Retriever-Setting erreicht es 67–94 % Genauigkeit und übertrifft GPT-4, GPT-3.5 und Claude in Genauigkeit wie Halluzinationsunterdrückung. Kombiniert Self-Instruct, Fine-Tuning und "Retriever-Aware

Training" (RAT) zur Anpassung an sich ändernde APIs. Führt **APIBench** ein — 1.645 APIs (925 HuggingFace, 696 TensorHub, 95 TorchHub).

ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs — Yujia Qin et al. (Tsinghua/OpenBMB), ICLR 2024 (Spotlight), arXiv:2307.16789. Führt **ToolBench** (Instruction-Tuning-Datensatz über 16.000+ RapidAPI-APIs) und einen Depth-First-Search-based Decision Tree (DFSDT) ein. Das resultierende ToolLLaMA erreicht ChatGPT-Niveau und generalisiert auf unbekannte APIs.

API-Bank: A Comprehensive Benchmark for Tool-Augmented LLMs — Minghao Li, Yingxiu Zhao, Bowen Yu et al. (Alibaba u. a.), EMNLP 2023, arXiv:2304.08244. Lauffähiges Evaluationssystem mit 73 API-Tools und 314 annotierten Tool-Nutzungs-Dialogen (753 API-Aufrufe); bewertet drei Fähigkeitsdimensionen: *Call*, *Retrieve+Call* und *Plan+Retrieve+Call*.

OpenAI Function Calling (Juni 2023) — Produkt-/API-Feature (kein Paper): strukturierte JSON-Ausgabe von Funktionsaufrufen; de-facto-Industriestandard, der Function Calling in die Breite trug. Weitergeführt in der Assistants API und Tool-Calling.

Code-as-Tool / Program-aided

PAL: Program-aided Language Models — Luyu Gao, Aman Madaan, Shuyan Zhou, Uri Alon et al. (CMU), ICML 2023, arXiv:2211.10435. LLM liest das Problem und erzeugt ein Programm als Zwischenschritt, die eigentliche Lösung wird an einen Python-Interpreter ausgelagert — deutliche Verbesserungen bei arithmetischem/symbolischem Reasoning.

Verwandt: Program of Thoughts (PoT), Code Interpreter (OpenAI), ToRA (Tool-integrated Reasoning Agent).

Instruction Tuning für Tool-Nutzung

ToolLLaMA (aus ToolLLM), Gorilla, GPT4Tools (Rui Yang et al., 2023, arXiv:2305.18752 — Self-Instruct für Tool-Nutzung), ToolAlpaca (Qiaoyu Tang et al., 2023, arXiv:2306.05301 — generalisiertes Tool-Learning mit ~3000 simulierten Fällen).

Agentische Frameworks / Multi-Tool-Orchestrierung

AutoGPT — Toran Bruce Richards / Significant Gravitas, März 2023 (Open-Source-Projekt, kein Paper). Autonomer GPT-4-Agent, der ein Ziel selbstständig in Teilaufgaben zerlegt und via Web-Browsing, Dateiverwaltung und Code-Ausführung abarbeitet; [Learn AI](#)

↗
löste die "Agentic-AI"-Welle 2023 aus.

AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation — Qingyun Wu, Gagan Bansal et al. (Microsoft), 2023, arXiv:2308.08155. Framework, in dem mehrere anpassbare, "conversable" Agenten miteinander (und mit Mensch/Tools) kommunizieren, um Aufgaben zu lösen. [arXiv](#)

↗
HuggingGPT (Yongliang Shen et al., 2023, arXiv:2303.17580) — ChatGPT als Controller, der Modelle aus dem Hugging-Face-Ökosystem orchestriert.

Reflexion (Noah Shinn et al., NeurIPS 2023) — verbale Selbstverstärkung/Selbstkritik für Agenten.

Model Context Protocol (MCP)

Model Context Protocol (MCP) — Anthropic, offiziell am 25. November 2024 als offener Standard veröffentlicht (Spec-Version 2024-11-05), mit SDKs für Python/TypeScript. Standardisierte Client-Server-Schnittstelle, die das "M×N-Integrationsproblem" löst (statt M×N Konnektoren nur je eine

Implementierung pro Seite — "USB-C für KI"). [Medium](#)



Server bieten drei Primitiven: *Prompts*, *Resources*, *Tools*; Kommunikation via JSON-RPC. Nach der Ankündigung von OpenAI und Google DeepMind übernommen (2025). MCP ist der derzeit wichtigste standardisierte Tool-Integrations-Layer.

B) RAG

Fundierende Architekturen

REALM: Retrieval-Augmented Language Model Pre-Training — Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasupat, Mingwei Chang (Google), ICML 2020. Integriert einen dichten Retriever bereits ins Pre-Training (maskiertes LM-Objektiv), end-to-end trainiert; einer der ersten retrieval-augmentierten LMs.

Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks — Patrick Lewis, Ethan Perez, Aleksandra Piktus et al. (Facebook AI/UCL), NeurIPS 2020, arXiv:2005.11401. **Das RAG-Originalpaper.** Kombiniert einen vortrainierten Retriever (DPR, Query-Encoder + Dokumentenindex) mit einem seq2seq-Generator; nutzt Maximum Inner Product Search (MIPS) für Top-K-Dokumente und behandelt diese als latente Variable. Prägt das Paradigma parametrisches + nicht-parametrisches Gedächtnis.

FID (Fusion-in-Decoder): Leveraging Passage Retrieval with Generative Models for Open Domain QA — Gautier Izacard, Edouard Grave (Facebook AI), EACL 2021. Enkodiert mehrere Passagen separat und fusioniert die Evidenz erst im Decoder; setzte SOTA auf QA-Benchmarks.

RETRO: Improving Language Models by Retrieving from Trillions of Tokens — Sebastian Borgeaud, Arthur Mensch et al. (DeepMind), ICML 2022, arXiv:2112.04426. Konditioniert ein autoregressives LM über *chunked cross-attention* auf aus einer 2-Billionen-Token-Datenbank abgerufene Chunks. Konkret erreicht ein **RETRO-Modell mit 7,5B Parametern** vergleichbare Leistung wie GPT-3 (175B) und Jurassic-1 (178B) auf dem Pile (Originalzitat: "With a 2 trillion token database, our Retrieval-Enhanced Transformer (Retro) obtains comparable performance to GPT-3 and Jurassic-1 on the Pile, despite using 25× fewer parameters").

Ergänzend: **kNN-LM** (Khandelwal et al., 2020), **Atlas** (Izacard et al., 2022), **In-Context RALM** (Ram et al., TACL 2023).

Taxonomie Naive / Advanced / Modular RAG (nach Gao et al. 2023)

Naive RAG: Standardpipeline Indexing → Retrieval → Generation (oft BM25/dense); Probleme: geringe Präzision, Halluzination, Redundanz. [arxiv](#)



Advanced RAG: Optimierung durch Pre-Retrieval (bessere Chunking-/Indexing-Strategien, Metadaten) und Post-Retrieval (Re-Ranking, Prompt-Kompression); dichtes semantisches Matching, Multi-Hop.

Modular RAG: zerlegt die Pipeline in konfigurierbare Module (Search-, Memory-, Tuning-, Task-Adapter-Module); [arxiv](#)



dominierendes Paradigma, ermöglicht iterative/adaptive Abläufe.

Bausteine: Chunking, Embeddings, Vektor-DBs

Chunking-Strategien (fixe Größe, semantisch, satzbasiert, Metadaten-bewusst); Embedding-Modelle (Sentence-BERT/Reimers & Gurevych 2019; BGE/M3, OpenAI/Cohere-Embeddings); Approximate Nearest Neighbor Search (HNSW, Malkov & Yashunin; FAISS, Johnson et al.).

Query-Transformation / Re-Ranking

HyDE: Precise Zero-Shot Dense Retrieval without Relevance Labels — Luyu Gao, Xueguang Ma, Jimmy Lin, Jamie Callan (CMU/Waterloo), ACL 2023, arXiv:2212.10496. Ein instruktionsfolgendes LLM erzeugt zunächst ein *hypothetisches Dokument* zur Query; dessen Embedding dient dann zur Suche realer Dokumente — starke Zero-Shot-Retrieval-Leistung ohne Relevanzlabels.

Weitere: Query Rewriting/Decomposition, Step-Back Prompting, RQ-RAG (Chan et al., 2024); Re-Ranking via Cross-Encoder, RankRAG (Yu et al., 2024), Re2G (Glass et al., NAACL 2022).

Selbstreflektierendes / korrigierendes / adaptives RAG

Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection — Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, Hannaneh Hajishirzi (UW/Allen AI/IBM), ICLR 2024 (Oral, Top 1%), arXiv:2310.11511. Ein einzelnes LM entscheidet *on-demand*, ob überhaupt abgerufen wird, und bewertet abgerufene Passagen und eigene Ausgaben über *Reflection Tokens*.

Corrective RAG (CRAG) — Shi-Qi Yan, Jia-Chen Gu, Yun Zhu, Zhen-Hua Ling (USTC), 2024, arXiv:2401.15884. Leichtgewichtiger Retrieval-Evaluator bewertet die Qualität abgerufener Dokumente und triggert je nach Confidence unterschiedliche Aktionen (u. a. Web-Suche als Fallback bei schlechter Retrieval-Qualität); mit bestehenden RAG-Ansätzen kombinierbar.

Adaptive-RAG — Soyeong Jeong, Jinheon Baek et al. (KAIST), NAACL 2024, arXiv:2403.14403. Passt die Retrieval-Strategie/-Tiefe dynamisch an die Fragekomplexität an (keine Abfrage / Single-Step / Multi-Step).

Active RAG (FLARE) — Zhengbao Jiang et al. (CMU), 2023, arXiv:2305.06983. Aktives, antizipatorisches Abrufen während der Generierung.

Graph RAG

From Local to Global: A Graph RAG Approach to Query-Focused Summarization — Darren Edge, Ha Trinh, Newman Cheng et al. (Microsoft Research), 2024, arXiv:2404.16130. Baut per LLM einen Entitäten-Wissensgraphen aus dem Korpus, erzeugt Community-Summaries und beantwortet damit *globale* Sensemaking-Fragen ("Was sind die Hauptthemen?"), an denen klassisches RAG scheitert. Auf zwei Korpora (Podcast-Transkripte ~1M Tokens; News ~1,7M Tokens) erzielte GraphRAG gegen konventionelles Vektor-RAG **Comprehensiveness-Win-Rates von 72-83 % ($p < .001$)** und Diversity-Win-Rates von 62-82 %. Verwandt: LightRAG, HippoRAG, GRAG.

Agentic RAG (Brücke zwischen A und B)

Agentic Retrieval-Augmented Generation: A Survey on Agentic RAG — Aditi Singh, Abul Ehtesham, Saket Kumar, Tala Talaei Khoei, 2025, arXiv:2501.09136. Taxonomie agentischer RAG-Architekturen entlang Reflection, Planning, Tool Use und Multi-Agent-Kollaboration. Kernidee: autonome Agenten steuern Retrieval iterativ, verschränken Suche und Reasoning und nutzen Tools — die Konvergenz der beiden Teilthemen A und B.

Reasoning RAG via System 1 or System 2: A Survey on Reasoning Agentic RAG — 2025, arXiv:2506.10408.

Long-Context vs. RAG Debatte

Retrieval Augmented Generation or Long-Context LLMs? A Comprehensive Study and Hybrid

Approach — Zhuowan Li, Cheng Li, Mingyang Zhang, Qiaozhu Mei, Michael Bendersky (Google), EMNLP 2024 (Industry Track). Long-Context-LLMs übertreffen RAG bei ausreichenden Ressourcen, aber RAG ist

deutlich kosteneffizienter; [Meilisearch](#)

↗

schlägt einen hybriden "Self-Route"-Ansatz vor.

Long Context vs. RAG for LLMs: An Evaluation and Revisits — 2025, arXiv:2501.01880. LC schlägt RAG meist bei Wikipedia-basierten Fragen; RAG hat Vorteile bei dialogbasierten/allgemeinen Anfragen. [arXiv](#)

↗

Kein "Silver Bullet".

Long-Context LLMs Meet RAG — Bowen Jin et al., 2024, arXiv:2410.05983. Zeigt umgekehrt-U-förmige Kurve: mehr abgerufene Passagen verbessern zunächst, verschlechtern dann durch Ablenkung ("Distraction"). [arxiv](#)

↗

Empirische Vergleiche / Benchmarks

A) Tool Use / Agenten

Berkeley Function Calling Leaderboard (BFCL) — Shishir G. Patil et al. (UC Berkeley), ICML 2025, laufend aktualisiert (aktuell v4). De-facto-Standard für Function-Calling-Evaluation: bewertet serielle und parallele Aufrufe über mehrere Programmiersprachen mittels **Abstract-Syntax-Tree-(AST)-Evaluation**, [Berkeley](#)

↗

die auf tausende Funktionen skaliert. v3 führte Multi-Turn/Multi-Step ein, v4 holistische agentische Evaluation. [Berkeley](#)

↗

Kernbefund: SOTA-LLMs meistern Single-Turn-Aufrufe, scheitern aber häufig an Gedächtnis, dynamischer Entscheidung und Long-Horizon-Reasoning.

ToolBench (aus ToolLLM) — Multi-Tool/Multi-Step über 16.000+ APIs.

API-Bank — s. o., drei Fähigkeitsdimensionen (73 Tools, 314 Dialoge, 753 API-Aufrufe).

Weitere: T-Eval (schrittweise Tool-Nutzung), ToolTalk (konversationell), MINT (Multi-Turn mit Feedback), GAIA (General AI Assistants Benchmark, Mialon et al. 2023).

B) RAG

RGB — Benchmarking Large Language Models in Retrieval-Augmented Generation — Jiawei Chen, Hongyu Lin, Xianpei Han, Le Sun (CAS), AAAI 2024, arXiv:2309.01431. Zweisprachiger (EN/ZH) Benchmark mit vier separaten Testbeds für vier grundlegende RAG-Fähigkeiten: **Noise Robustness, Negative Rejection, Information Integration, Counterfactual Robustness**. [AAAI](#)

↗

Befund: LLMs sind einigermaßen robust gegen Rauschen, haben aber erhebliche Schwächen bei Negative Rejection, Informationsintegration und dem Umgang mit falschen Informationen. [Semantic Scholar](#)

↗

RAGAS: Automated Evaluation of Retrieval Augmented Generation — Shahul Es, Jithin James, Luis Espinosa-Anke, Steven Schockaert (Exploding Gradients/Cardiff), EACL 2024 (Demos), arXiv:2309.15217. Referenzfreies Evaluationsframework mit Metriken *Faithfulness, Answer Relevance, Context Relevance* [arXiv](#)

↗

(bzw. Context Precision/Recall) — ohne menschliche Ground-Truth-Annotationen. [arXiv](#)

↗

Weit verbreitet in der Praxis.

Weitere: **ARES** (Saad-Falcon et al., 2024, arXiv:2311.09476), **RAGBench** (Friel et al., 2024, arXiv:2407.11005), TruLens, "Searching for Best Practices in RAG" (Wang et al., 2024).

Aktuelle Entwicklungen 2024–2026

Standardisierung der Tool-Integration: MCP (Anthropic, Nov 2024) hat sich rasant zum Branchenstandard entwickelt und wurde 2025 von OpenAI und Google DeepMind übernommen; laufende Spec-Revisionen (2025-03, 2025-06, 2025-11) erweitern Transport (HTTP/SSE), Auth und Elicitation.

Agentic RAG als Konvergenz: RAG wird von statischen Pipelines zu agentischen Architekturen (Adaptive-RAG, Self-RAG, CRAG, Search-R1, Search-o1); RL-basierte Suchfähigkeit (R1-Searcher, ZeroSearch, Search-R1) ist ein 2025er Trend.

GraphRAG-Ökosystem: Microsoft GraphRAG (2024), LightRAG, HippoRAG, MiniRAG — Wissensgraph-gestützte Ansätze für globale Fragen und Multi-Hop-Reasoning.

Function-Calling-Reife: BFCL v3/v4 verschieben den Fokus von einzelnen Aufrufen zu Multi-Turn-/agentischer Evaluation; native Tool-Calling-APIs bei allen großen Anbietern.

Long-Context vs. RAG: Mit 1M+-Token-Fenstern (Gemini 1.5/2.x, GPT-4.1) verschiebt sich die Debatte zu hybriden Routing-Ansätzen; RAG bleibt aus Kosten-, Aktualitäts- und Edge-Gründen relevant.

Grenzen & Risiken

A) Tool Use / Agenten

Halluzinierte/fehlerhafte API-Aufrufe: falsche Argumente, nicht existente Funktionen (Motivation von Gorilla, BFCL).

Fehler-Fortpflanzung und Long-Horizon-Schwäche: Agenten scheitern an Gedächtnis, dynamischer Entscheidung und langen Aufgabenketten (BFCL-Befund).

Sicherheit: Prompt Injection über Tool-Ausgaben, unkontrollierte Aktionen autonomer Agenten, MCP-Server-Sicherheit.

Zuverlässigkeit/Kosten: viele API-Roundtrips, Latenz, Nicht-Determinismus.

B) RAG

Seven Failure Points When Engineering a RAG System — Scott Barnett et al. (Deakin University), CAIN 2024, arXiv:2401.05856. Sieben Fehlerpunkte: (FP1) fehlender Inhalt, (FP2) verpasste Top-Dokumente (Ranking), (FP3) nicht im Kontext (Konsolidierung), (FP4) nicht extrahiert, (FP5) falsches Format, (FP6) falsche Spezifität, (FP7) unvollständig. Kernbotschaft: RAG-Robustheit entsteht erst im Betrieb, nicht am Reißbrett. [arxiv](#)

↗

Lost in the Middle — Nelson F. Liu et al. (Stanford), TACL 2024, arXiv:2307.03172. Leistung folgt einer U-Kurve: Modelle nutzen Information am Anfang/Ende des Kontexts am besten, in der Mitte deutlich schlechter [arXiv](#)

↗

— relevant für Chunk-Reihenfolge und Kontextgröße.

Halluzination trotz Retrieval: LLMs schwach bei *Negative Rejection* (Ablehnung, wenn Antwort fehlt) und *Counterfactual Robustness* (RGB-Befund).

Retrieval-Qualität als Flaschenhals: verrauschte/irrelevante Passagen lenken ab (Distraction, inverted-U); Präferenz-Gap zwischen Retriever und LLM. [arxiv](#)



Weitere: Chunking-Artefakte, veraltete Indizes, Bias-Verstärkung durch Retrieval, Kosten der Web-Suche-Fallbacks (CRAG).

Praxis-Ecke (Frameworks / Tools)

Tool-Use / Agent-Frameworks

LangChain — verbreitetstes Orchestrierungsframework; Agents, Tool-Abstraktion, ReAct-Implementierungen.

LlamaIndex — datenzentriert, mit Agent-/Tool-Abstraktionen und Query-Engines.

AutoGPT (Significant Gravitas) — autonomer Agent, Proof-of-Concept der Agentic-Welle.

Microsoft AutoGen — Multi-Agent-Konversationsframework (arXiv:2308.08155).

CrewAI (João Moura, 2023) — rollenbasierte Multi-Agent-Orchestrierung (Role/Goal/Backstory; Crews & Flows). [noze](#)



Anthropic MCP-Ökosystem — offener Standard + vorgefertigte Server (Google Drive, Slack, GitHub, Git, Postgres, Puppeteer).

OpenAI Assistants / Function Calling API — native strukturierte Tool-Aufrufe, Code Interpreter, File Search.

Weitere: LangGraph (Graph-basierte Agent-Workflows), Semantic Kernel (Microsoft), Haystack Agents.

RAG-Frameworks & Infrastruktur

Frameworks: LangChain, LlamaIndex, **Haystack** (deepset), RAG Foundry, DSPy.

Vektor-Datenbanken: **Pinecone** (managed), **Weaviate**, **Chroma** (leichtgewichtig/lokal), **Milvus** (skalierbar, OSS), **Qdrant**, **pgvector** (PostgreSQL-Erweiterung), FAISS (Bibliothek).

Embedding-Anbieter: OpenAI, Cohere, Voyage AI, Google, sowie Open-Source (Sentence-Transformers, BGE/BAAI, E5, Nomic).

Evaluation: RAGAS, ARES, TruLens, LangSmith, Phoenix (Arize).

Empfehlungen zur Präsentationsgliederung

Einstieg / Motivation (2 Folien): Warum Augmentierung? Grenzen parametrischen Wissens (Halluzination, Aktualität, fehlende Handlungsfähigkeit). Zwei Antworten: Wissen holen (RAG) vs. Handeln (Tools).

Block A — Tool Use / Agenten:

Survey-Fundament (Mialon et al.).

Paradigmen-Evolution: MRKL → ReAct → Toolformer → Function Calling (OpenAI) → Gorilla/ToolLLM.

Agenten & Orchestrierung: AutoGPT/AutoGen/CrewAI; MCP als Standard.

Benchmarks: BFCL, API-Bank, ToolBench.

Block B — RAG:

Survey-Fundament & Taxonomie (Gao et al.: Naive/Advanced/Modular).

Architektur-Historie: REALM → RAG (Lewis) → FiD → RETRO.

Pipeline-Bausteine: Chunking, Embeddings, Vektor-DBs, HyDE, Re-Ranking.

Fortgeschritten: Self-RAG, CRAG, Adaptive-RAG, GraphRAG.

Benchmarks & Evaluation: RGB, RAGAS.

Brücke — Agentic RAG: Konvergenz von A und B; Long-Context-vs-RAG-Debatte.

Grenzen & Risiken: Seven Failure Points, Lost in the Middle, Sicherheit/Prompt Injection.

Praxis-Ecke: Framework-/Tool-Landkarte (Entscheidungshilfe: wann LangChain vs. LlamaIndex vs. Haystack; welche Vektor-DB).

Fazit & Ausblick: Standardisierung (MCP), agentische Systeme, hybride Long-Context/RAG-Architekturen.

Schwellen, die die Empfehlung ändern würden: Wenn der interne Use Case primär *statische, kleine* Dokumentbestände betrifft, Long-Context-LLM statt RAG evaluieren. Wenn viele heterogene interne Systeme angebunden werden müssen, MCP früh als Integrations-Layer setzen. Wenn Multi-Hop-/Sensemaking-Fragen dominieren, GraphRAG statt Naive RAG prüfen.

Caveats

Quellencharakter: AutoGPT und CrewAI sind Open-Source-Projekte, keine peer-reviewten Paper; Daten (AutoGPT März 2023; CrewAI initiale PyPI-Veröffentlichung Dez. 2023) stammen aus Projekt-Repos/Sekundärquellen und variieren leicht je nach Quelle.

Versionsdynamik: MCP-Spezifikation und BFCL werden fortlaufend aktualisiert (MCP-Basis 2024-11-05, BFCL aktuell v4); genaue Feature-Stände können sich seit Redaktionsschluss (Aug. 2026) verschoben haben.

Zitationszahlen und Venue-Zuordnung: Einige Paper erschienen zuerst als arXiv-Preprint und später in Konferenzen/Journalen (z. B. Gorilla arXiv 2023 → NeurIPS 2024; Self-RAG/CRAG als ICLR-2024-Beiträge); Jahresangaben können je nach Zitierweise (Preprint vs. Proceedings) differieren.

Konfligierende Befunde Long-Context vs. RAG: Die Studienlage ist uneinheitlich (teils LC > RAG, teils umgekehrt); Ergebnisse hängen stark von Modellkapazität, Aufgabentyp und Retrieval-Qualität ab — für interne Entscheidungen ist eine eigene Evaluation auf dem konkreten Use Case ratsam.

Nicht abschließend: Der Überblick priorisiert hochzitierte/kanonische Arbeiten; das Feld (insb. Agentic RAG, RL-basierte Suche) entwickelt sich schnell, sodass neuere Einzelarbeiten bewusst nur exemplarisch genannt sind.